

1 Create Database

```
DROP DATABASE IF EXISTS de_week2;  
CREATE DATABASE de_week2;  
USE de_week2;
```

2 Create Tables

```
DROP TABLE IF EXISTS order_items;  
DROP TABLE IF EXISTS orders;  
DROP TABLE IF EXISTS products;  
DROP TABLE IF EXISTS customers;
```

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    full_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100),  
    state CHAR(2),  
    signup_date DATE  
);
```

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    category VARCHAR(50) NOT NULL,  
    unit_price DECIMAL(10,2) NOT NULL  
);
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    channel VARCHAR(20),  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

```
CREATE TABLE order_items (
    order_id INT,
    line_no INT,
    product_id INT,
    quantity INT NOT NULL,
    PRIMARY KEY (order_id, line_no),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

3 Insert Extended Customers (20 rows)

```
INSERT INTO customers VALUES
(1, 'Asha Patel', 'asha@gmail.com', 'IL', '2025-01-10'),
(2, 'Ben Moore', 'ben@gmail.com', 'TX', '2025-02-03'),
(3, 'Chen Li', 'chen@gmail.com', 'CA', '2025-02-20'),
(4, 'Dina Khan', 'dina@gmail.com', 'NY', '2025-03-01'),
(5, 'Evan Reed', NULL, 'TX', '2025-03-05'),
(6, 'Farah Ali', 'farah@gmail.com', 'IL', '2025-01-25'),
(7, 'George Park', 'george@gmail.com', 'CA', '2025-02-14'),
(8, 'Hannah Scott', NULL, 'FL', '2025-02-18'),
(9, 'Ibrahim Noor', 'ibrahim@gmail.com', 'TX', '2025-03-02'),
(10, 'Julia Adams', 'julia@gmail.com', 'WA', '2025-03-03'),
(11, 'Kevin Tran', 'kevin@gmail.com', 'CA', '2025-03-07'),
(12, 'Lina Gomez', 'lina@gmail.com', 'FL', '2025-03-09'),
(13, 'Mohammed Saleh', NULL, 'IL', '2025-03-10'),
(14, 'Nina Kapoor', 'nina@gmail.com', 'TX', '2025-03-12'),
(15, 'Oscar Diaz', 'oscar@gmail.com', 'AZ', '2025-03-15'),
(16, 'Priya Shah', 'priya@gmail.com', 'CA', '2025-03-17'),
(17, 'Quinn Brooks', 'quinn@gmail.com', 'WA', '2025-03-18'),
(18, 'Ravi Menon', NULL, 'NY', '2025-03-19'),
(19, 'Sara Lee', 'sara@gmail.com', 'FL', '2025-03-20'),
(20, 'Tom Walker', 'tom@gmail.com', 'TX', '2025-03-21');
```

4 Insert Extended Products (15 rows)

```
INSERT INTO products VALUES
(10,'Spark Plug','Auto',9.99),
(11,'Oil Filter','Auto',12.50),
(12,'Tire','Auto',89.00),
(13,'Brake Pad','Auto',45.00),
(14,'Engine Oil','Auto',35.00),
(20,'Notebook','Office',3.25),
(21,'Pen','Office',1.10),
(22,'Desk Chair','Office',120.00),
(23,'Monitor','Office',250.00),
(24,'Keyboard','Office',40.00),
(30,'Water Bottle','Lifestyle',14.00),
(31,'Backpack','Lifestyle',60.00),
(32,'Running Shoes','Lifestyle',95.00),
(33,'Yoga Mat','Lifestyle',25.00),
(34,'Smart Watch','Lifestyle',199.00);
```

5 Insert Extended Orders (40 rows)

Mix of:

- COMPLETED
- CANCELLED
- PENDING
- WEB / MOBILE

```
INSERT INTO orders VALUES
(101,1,'2025-03-10','COMPLETED','WEB'),
(102,1,'2025-03-12','CANCELLED','WEB'),
(103,2,'2025-03-11','COMPLETED','MOBILE'),
(104,3,'2025-03-15','COMPLETED','WEB'),
(105,4,'2025-03-16','PENDING','MOBILE'),
(106,5,'2025-03-17','COMPLETED','WEB'),
(107,6,'2025-03-18','COMPLETED','WEB'),
```

(108,7,'2025-03-18','CANCELLED','MOBILE'),
(109,8,'2025-03-19','COMPLETED','WEB'),
(110,9,'2025-03-20','COMPLETED','MOBILE'),
(111,10,'2025-03-21','PENDING','WEB'),
(112,11,'2025-03-22','COMPLETED','WEB'),
(113,12,'2025-03-23','COMPLETED','MOBILE'),
(114,13,'2025-03-24','COMPLETED','WEB'),
(115,14,'2025-03-25','CANCELLED','MOBILE'),
(116,15,'2025-03-26','COMPLETED','WEB'),
(117,16,'2025-03-27','COMPLETED','WEB'),
(118,17,'2025-03-28','PENDING','MOBILE'),
(119,18,'2025-03-29','COMPLETED','WEB'),
(120,19,'2025-03-30','COMPLETED','WEB'),
(121,20,'2025-03-30','COMPLETED','MOBILE'),
(122,3,'2025-03-31','COMPLETED','WEB'),
(123,5,'2025-04-01','COMPLETED','WEB'),
(124,7,'2025-04-01','COMPLETED','MOBILE'),
(125,9,'2025-04-02','CANCELLED','WEB'),
(126,11,'2025-04-02','COMPLETED','WEB'),
(127,13,'2025-04-03','COMPLETED','MOBILE'),
(128,15,'2025-04-04','COMPLETED','WEB'),
(129,17,'2025-04-05','COMPLETED','WEB'),
(130,19,'2025-04-06','PENDING','MOBILE'),
(131,2,'2025-04-07','COMPLETED','WEB'),
(132,4,'2025-04-07','COMPLETED','WEB'),
(133,6,'2025-04-08','COMPLETED','MOBILE'),
(134,8,'2025-04-08','COMPLETED','WEB'),
(135,10,'2025-04-09','CANCELLED','WEB'),
(136,12,'2025-04-09','COMPLETED','WEB'),
(137,14,'2025-04-10','COMPLETED','MOBILE'),
(138,16,'2025-04-10','COMPLETED','WEB'),
(139,18,'2025-04-11','COMPLETED','WEB'),
(140,20,'2025-04-12','COMPLETED','MOBILE');

6 Insert Extended Order Items (Multi-line orders)

```
INSERT INTO order_items VALUES
(101,1,10,2),(101,2,11,1),
(103,1,12,4),
(104,1,20,10),(104,2,21,20),
(105,1,30,1),
(106,1,13,2),
(107,1,14,1),
(109,1,22,1),
(110,1,32,2),
(112,1,23,1),
(113,1,24,3),
(114,1,31,2),
(116,1,33,1),
(117,1,34,1),
(119,1,10,5),
(120,1,12,2),
(121,1,14,1),
(122,1,22,2),
(123,1,30,4),
(124,1,32,1),
(126,1,23,2),
(127,1,11,3),
(128,1,13,2),
(129,1,34,1),
(131,1,21,10),
(132,1,12,1),
(133,1,24,1),
(134,1,31,2),
(136,1,10,1),
(137,1,32,2),
(138,1,33,3),
(139,1,14,2),
(140,1,34,1);
```

SQL in Data Engineering + Relational Basics + Core Query Patterns

SQL is used to turn raw operational data into analytics-ready data. In most systems, data moves through layers:

Raw landing data → staging tables → cleaned tables → enriched tables → aggregated marts → dashboards / ML features

- In raw and staging, SQL is used for **sanity checks**: row counts, duplicates, missing keys, null rates.
- In clean layers, SQL is used for **standardization**: trimming strings, fixing invalid values, applying consistent formats, removing outliers.
- In enriched layers, SQL is used to **connect datasets**: joining transactions to customers, products, channels, locations.
- In aggregated marts, SQL is used to **reduce detail into KPIs**: revenue by day, orders by channel, customers by lifetime spend.
- In analytics consumption, SQL is used for **final slicing** and fast retrieval.

Relational structure using the dataset

There are four tables and each has a clear purpose:

customers: who is buying

orders: order header (one row per order)

order_items: order lines (one row per product per order)

products: product catalog / reference data

Relationships:

- customers → orders (one-to-many)
- orders → order_items (one-to-many)
- products → order_items (one-to-many)

Grain

Grain means what one row represents.

- customers: 1 row = 1 customer
- orders: 1 row = 1 order
- order_items: 1 row = 1 order line

Grain = what one row represents.

Consider these simplified tables:

customers

customer_id	full_name	state
1	Asha Patel	IL
2	Ben Moore	TX
3	Chen Li	CA

Grain: 1 row = 1 customer.

orders

order_id	customer_id	order_date	status
101	1	2025-03-10	COMPLETED
102	1	2025-03-12	CANCELLED
103	2	2025-03-11	COMPLETED

Grain: 1 row = 1 order.

order_items

order_id	product_id	quantity
101	10	2
101	11	1

103	12	4
-----	----	---

Grain: 1 row = 1 order line.

Understanding grain prevents revenue duplication later

When you join tables, the grain often changes. If you join orders to order_items, each order becomes multiple rows. That is expected, but you must handle it correctly using aggregation when needed.

Primary key and foreign key

- customers.customer_id is a primary key (unique identifier)
- orders.customer_id is a foreign key (points to customers)
- order_items.order_id points to orders
- order_items.product_id points to products

If foreign keys are missing or invalid:

- INNER JOIN can drop records
- Aggregations can change
- KPIs can be wrong

Logical query execution order (how databases process queries)

1. FROM / JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
7. LIMIT

This matters because:

- WHERE filters rows before grouping
- HAVING filters groups after aggregation

SELECT patterns (MySQL)

Basic:

```
SELECT customer_id, full_name, state
FROM customers;
```

Derived columns:

```
SELECT
    full_name,
    CONCAT(state, '-USA') AS region
FROM customers;
```

CASE for classification:

```
SELECT
    order_id,
    status,
    CASE
        WHEN status = 'COMPLETED' THEN 'Closed'
        WHEN status = 'CANCELLED' THEN 'Lost'
        ELSE 'Open'
    END AS order_group
FROM orders;
```

Dates:

```
SELECT
    order_id,
    order_date,
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month
FROM orders;
```

WHERE patterns

Equality:

```
SELECT *  
FROM orders  
WHERE status = 'COMPLETED';
```

IN:

```
SELECT *  
FROM customers  
WHERE state IN ('TX', 'CA');
```

BETWEEN:

```
SELECT *  
FROM orders  
WHERE order_date BETWEEN '2025-04-01' AND '2025-04-30';
```

NULL handling:

```
SELECT *  
FROM customers  
WHERE email IS NULL;
```

Do not use:

```
WHERE email = NULL;
```

exercises

1. Show all customers from TX.
2. Show all orders from WEB channel.
3. Show orders in April 2025.
4. Show customers with missing email.
5. Show distinct order statuses.
6. Add a derived column `order_month` from `order_date`.
7. Classify orders as Closed/Lost/Open using CASE.

Aggregations + GROUP BY + HAVING + Conditional Aggregation

Aggregation turns detail into summaries (KPIs).

1. Why Aggregations Exist

Raw:

order_id	quantity	price
101	2	10
101	1	12

Business wants:

Order 101 → total revenue = ?

Aggregation converts detail to summary

Example detail:

order_items has multiple rows per order. Business usually wants:

- revenue per order
- revenue per customer
- revenue per state
- revenue per day/month

COUNT variations

- COUNT(*) counts rows
- COUNT(col) counts only non-null values

```
SELECT COUNT(*) AS total_customers,  
       COUNT(email) AS customers_with_email  
FROM customers;
```

GROUP BY changes grain

Orders per customer:

```
SELECT customer_id, COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id;
```

SUM revenue per order (join + aggregate)

```
SELECT
    oi.order_id,
    SUM(oi.quantity * p.unit_price) AS order_revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY oi.order_id;
```

HAVING (filtering aggregated groups)

Customers with more than 2 orders:

```
SELECT customer_id, COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 2;
```

Conditional aggregation (very common in marts)

Orders by status per customer:

```
SELECT
    customer_id,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN status='COMPLETED' THEN 1 ELSE 0 END) AS
completed_orders,
    SUM(CASE WHEN status='CANCELLED' THEN 1 ELSE 0 END) AS
cancelled_orders
FROM orders
GROUP BY customer_id;
```

Multi-dimensional grouping

Orders by channel + status:

```
SELECT channel, status, COUNT(*) AS cnt
FROM orders
GROUP BY channel, status;
```

exercises

1. Count orders per status.
2. Count orders per channel.
3. Count orders per channel per status.
4. Revenue per order.
5. Top 5 orders by revenue.
6. Customers with more than 3 completed orders.
7. Daily revenue for completed orders.
8. Revenue per category.

Joins + Data Integrity + Cardinality + Reconciliation Queries

Joins combine datasets. In DE, joins are also used to detect data issues.

1. INNER JOIN Mechanics

customers:

customer_id	name
1	Asha
2	Ben

orders:

order_id	customer_id
101	1

103	2
104	99

INNER JOIN:

```
SELECT *
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id;
```

Result excludes order 104.

2. LEFT JOIN Mechanics

```
SELECT *
FROM orders o
LEFT JOIN customers c
ON o.customer_id = c.customer_id;
```

Order 104 remains.
customer columns become NULL.

This is how you detect broken data.

3. Join Multiplication Problem

If products table has duplicates:

product_id	price
10	9.99
10	9.99

Joining will double revenue.

Always check uniqueness before joining.

4. Multi-Table Join Revenue Walkthrough

order_items:

order_id	product_id	quantity
101	10	2

products:

product_id	price
10	10

Join result:

order_id	quantity	price	revenue
101	2	10	20

Cardinality and row multiplication

- customers → orders is one-to-many (expected increase in rows)
- orders → order_items is one-to-many (expected increase in rows)
If the “one” side isn’t truly unique (duplicates), revenue will inflate.

Duplicate check pattern:

```
SELECT product_id, COUNT(*) AS cnt
FROM products
GROUP BY product_id
HAVING COUNT(*) > 1;
```

exercises

1. Join orders + customers and show order_id, name, state, status, channel.
2. Revenue per customer (completed only).
3. Revenue per state (completed only).
4. Customers with no orders.

5. Orders without items.
6. Products never sold.
7. Customers who used both WEB and MOBILE.

Customers who used both channels:

```
SELECT o.customer_id
FROM orders o
GROUP BY o.customer_id
HAVING COUNT(DISTINCT o.channel) = 2;
```

End-to-End Lab: Build Enriched Orders + Customer Mart + DQ Metrics

Goal tables:

- order_enriched: 1 row per order with revenue + units
- customer_mart: 1 row per customer with KPIs
- dq_metrics: daily health checks

Lab Step 1 — order_enriched

```
DROP TABLE IF EXISTS order_enriched;
```

```
CREATE TABLE order_enriched AS
SELECT
  o.order_id,
  o.customer_id,
  o.order_date,
  o.status,
  o.channel,
  COALESCE(SUM(oi.quantity * p.unit_price), 0) AS revenue,
  COALESCE(SUM(oi.quantity), 0) AS total_units
FROM orders o
LEFT JOIN order_items oi ON o.order_id = oi.order_id
LEFT JOIN products p ON oi.product_id = p.product_id
GROUP BY o.order_id, o.customer_id, o.order_date, o.status, o.channel;
```

Validation:

```
SELECT (SELECT COUNT(*) FROM orders) AS orders_cnt,  
       (SELECT COUNT(*) FROM order_enriched) AS enriched_cnt;
```

Lab Step 2 — customer_mart

```
DROP TABLE IF EXISTS customer_mart;
```

```
CREATE TABLE customer_mart AS  
SELECT  
    c.customer_id,  
    c.full_name,  
    c.state,  
    COUNT(e.order_id) AS total_orders,  
    SUM(CASE WHEN e.status='COMPLETED' THEN 1 ELSE 0 END) AS  
completed_orders,  
    SUM(CASE WHEN e.status='CANCELLED' THEN 1 ELSE 0 END) AS  
cancelled_orders,  
    SUM(CASE WHEN e.status='COMPLETED' THEN e.revenue ELSE 0 END) AS  
lifetime_revenue,  
    MIN(e.order_date) AS first_order_date,  
    MAX(e.order_date) AS last_order_date  
FROM customers c  
LEFT JOIN order_enriched e ON c.customer_id = e.customer_id  
GROUP BY c.customer_id, c.full_name, c.state;
```

Add avg_completed_order_value:

```
ALTER TABLE customer_mart ADD COLUMN avg_completed_order_value  
DECIMAL(10,2);
```

```
UPDATE customer_mart  
SET avg_completed_order_value =  
    CASE  
        WHEN completed_orders = 0 THEN 0  
        ELSE lifetime_revenue / completed_orders  
    END;
```

Lab Step 3 — dq_metrics

```
DROP TABLE IF EXISTS dq_metrics;
```

```
CREATE TABLE dq_metrics AS
SELECT
  CURDATE() AS run_date,
  (SELECT COUNT(*) FROM customers) AS customer_count,
  (SELECT COUNT(*) FROM orders) AS order_count,
  (SELECT COUNT(*) FROM order_items) AS order_item_count,
  (SELECT COUNT(*) FROM customers WHERE email IS NULL) AS
null_email_customers,
  (SELECT COUNT(*)
   FROM orders o
   LEFT JOIN customers c ON o.customer_id = c.customer_id
   WHERE c.customer_id IS NULL) AS orphan_orders,
  (SELECT COUNT(*)
   FROM orders o
   LEFT JOIN order_items oi ON o.order_id = oi.order_id
   WHERE oi.order_id IS NULL) AS orders_without_items;
```

exercises (mart queries)

1. Top 5 customers by lifetime_revenue.
2. Revenue by state (from customer_mart).
3. Cancellation rate by channel (from orders).
4. Revenue by category (completed only).
5. Find customers with lifetime_revenue = 0 but total_orders > 0 (hint: cancelled/pending only).